



IILM UNIVERSITY

C / C++ Programming Internship

Internship Completion Presentation

STUDENT NAME	Hammad Ahmad Khan
ENROLLMENT NUMBER	25SCS1003005449
PROGRAM & SEMESTER	B.Tech CSE — 3rd Semester
ORGANIZATION	Alfido Tech

Agenda

1

Internship Overview

About Alfido Tech and the internship structure

2

Task 2 — Algorithms & Problem Solving

Sorting, searching, and dynamic programming in C

3

Task 3 — System Programming

File I/O and process creation in C

4

Task 4 — C++ OOP & STL Project

Library Management System with classes, inheritance & STL

5

Skills Gained

Technical and professional takeaways

6

Conclusion

Reflection and what's next

Internship Overview

About the Internship

A C/C++ Programming Internship at Alfido Tech, focused on strengthening core computer science fundamentals through hands-on, task-based learning — covering data structures, algorithms, system-level programming, and object-oriented design in C++.

Internship Structure

Four independent tasks were provided, covering data structures, algorithms, system programming, and a full C++ OOP project. This presentation covers Tasks 2, 3, and 4, which were selected and submitted for evaluation.

All source code, README documentation, and build instructions are **publicly available on GitHub** for each task.

02

Algorithms & Problem Solving

03

System Programming / File I/O

04

C++ OOP & STL Project

TASK 02

Algorithms & Problem Solving

Goal

Solve algorithmic problems demonstrating sorting, searching, and dynamic programming approaches — implementing at least 3 problems with test cases and runtime measurement across input sizes.



Merge Sort

$O(n \log n)$ — guaranteed performance in every case via divide-and-merge



Binary Search

$O(\log n)$ — halves the search space every step on sorted data



0/1 Knapsack

Dynamic Programming — guarantees the true optimal value, unlike greedy

Deliverables

- ✓ Three separate C source files — one per algorithm
- ✓ Test cases proving correctness on fixed inputs
- ✓ Runtime benchmarks across multiple input sizes
- ✓ A written report on algorithm choice and complexity

Repository: `alfido-task2-algorithms`

Task 2 — Benchmark Results

Runtime measured across increasing input sizes to validate theoretical time complexity

Merge Sort

$O(n \log n)$

0.015s

at 100,000 elements

Binary Search

$O(\log n)$

0.000002s

at 10,000,000 elements

0/1 Knapsack

$O(n \times W)$

0.0037s

at 1,000 items

Key Takeaway

Binary Search stayed near-instant even at a 10,000x larger input — a direct, measured demonstration of why $O(\log n)$ scales so differently from $O(n)$ or $O(n \log n)$. Greedy selection was deliberately avoided for Knapsack, since it cannot guarantee the optimal answer; Dynamic Programming was used instead to build the mathematically correct solution.

TASK 03

System Programming

File I/O & Process Creation



What was built

A Simple Command Shell in C — reads a command, executes it as a new operating system process, and writes a timestamped log entry to a file for every command run.

Process Creation

Each typed command runs as a separate child process via `system()`

Persistent Logging

Every command, timestamp, and exit code is written to `shell_log.txt`

Graceful Error Handling

Invalid commands, empty input, and file errors never crash the shell

Design Decision

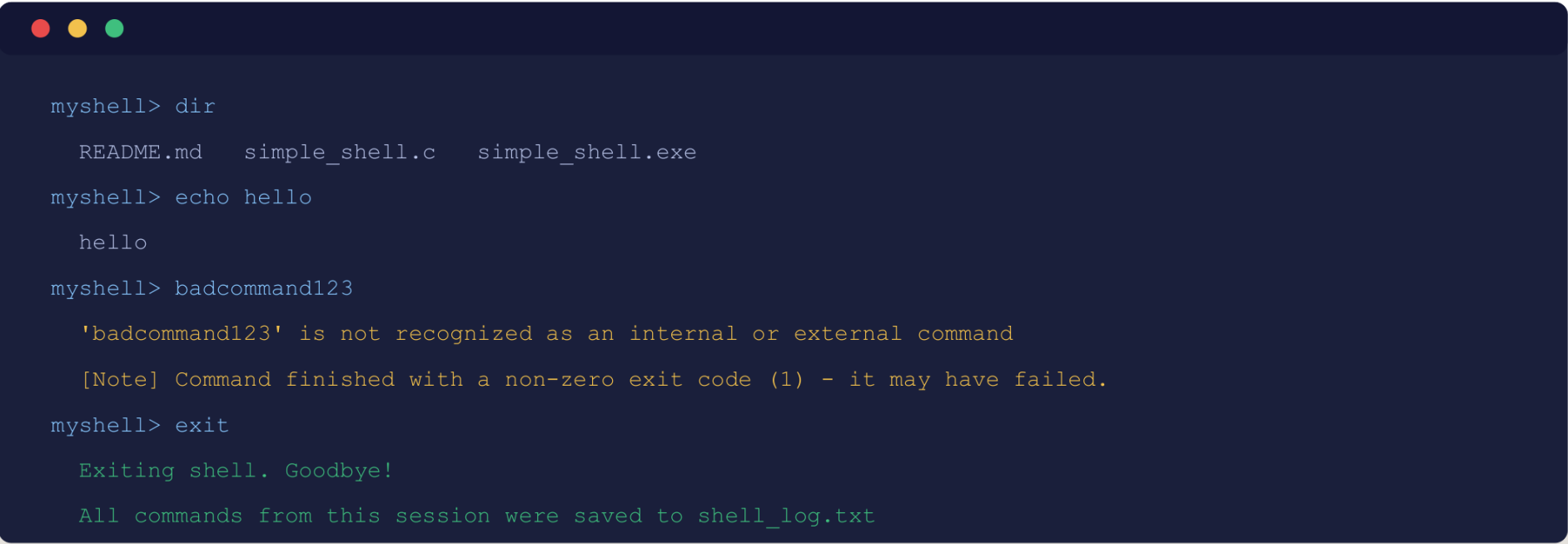
`fork()` and `exec()` — the classic Unix process-creation calls — do not exist on Windows. Since this was developed and tested on Windows, `system()` was used instead: a portable, standard C library function that works identically across Windows, Linux, and Mac.

Edge Cases Documented

- Unrecognized command
- Blank / empty input
- Log file fails to open
- Trailing newline in input

Task 3 — Sample Run

Terminal walkthrough demonstrating process execution, error handling, and logging



```
myshell> dir
  README.md  simple_shell.c  simple_shell.exe
myshell> echo hello
hello
myshell> badcommand123
'badcommand123' is not recognized as an internal or external command
[Note] Command finished with a non-zero exit code (1) - it may have failed.
myshell> exit
Exiting shell. Goodbye!
All commands from this session were saved to shell_log.txt
```

TASK 04

C++ Project — OOP & STL

Library Management System

A console-based C++ application demonstrating classes, inheritance, polymorphism, STL containers, and file persistence — built to specification with a Makefile and a hand-written unit test suite.



Classes & Inheritance

Book base class; EBook extends it with a virtual display() for polymorphism



STL Containers

vector for ordered storage, map for $O(\log n)$ lookup by book ID



File Persistence

Library data auto-saves and reloads — nothing lost between runs

Project Structure

Book.h

Book & EBook classes (inheritance)

Library.h

Library class — vector + map, save/load

main.cpp

Interactive menu program

test_library.cpp

13 unit tests

Makefile

One-command build automation

Repository: `library-management-system-cpp`

Task 4 — Testing & Validation

Correctness was verified with a hand-written unit test suite, not just manual runs



13 / 13

Unit Tests Passing

Covers core logic, edge cases, and file round-tripping



Core Functionality

Adding books, finding by ID, correct data returned



Edge Cases

Searching a missing ID, issuing an already-issued book



State Changes

Issue/return correctly flips availability status



File Persistence

Save-then-reload restores the exact same data

Skills Gained



Technical / Language

- C & C++ Programming
- Data Structures & STL Containers
- Algorithm Design & Complexity Analysis
- Memory Management (pointers, malloc/free)
- File I/O & Process Handling



Tools & Workflow

- Git & GitHub Version Control
- GCC / G++ Compilation & Makefiles
- Debugging & Runtime Benchmarking
- Technical Documentation (READMEs)



Professional

- Problem-Solving
- Self-Directed Learning
- Technical Communication
- Handling Ambiguity & Edge Cases

Conclusion

Thank You

Completing Tasks 2, 3, and 4 of the Alfido Tech C/C++ Programming Internship strengthened my grasp of core CS fundamentals — from algorithmic complexity to system-level programming to structured OOP design in C++. This foundation directly supports my continued path toward Cloud Engineering.

Hammad Ahmad Khan

B.Tech CSE, 3rd Semester — IILM University

C/C++ Programming Intern — Alfido Tech